

- Ostrowski, A.M. 1966, *Solutions of Equations and Systems of Equations*, 2nd ed. (New York: Academic Press), Chapter 12.
- Ridders, C.J.F. 1979, "A New Algorithm for Computing a Single Root of a Real Continuous Function," *IEEE Transactions on Circuits and Systems*, vol. CAS-26, pp. 979–980.[1]

9.3 Van Wijngaarden-Dekker-Brent Method

While secant and false position formally converge faster than bisection, one finds in practice pathological functions for which bisection converges more rapidly. These can be choppy, discontinuous functions, or even smooth functions if the second derivative changes sharply near the root. Bisection always halves the interval, while secant and false position can sometimes spend many cycles slowly pulling distant bounds closer to a root. Ridders' method does a much better job, but it too can sometimes be fooled. Is there a way to combine superlinear convergence with the sureness of bisection?

Yes. We can keep track of whether a supposedly superlinear method is actually converging the way it is supposed to, and, if it is not, we can intersperse bisection steps so as to guarantee *at least* linear convergence. This kind of super-strategy requires attention to bookkeeping detail, and also careful consideration of how round-off errors can affect the guiding strategy. Also, we must be able to determine reliably when convergence has been achieved.

An excellent algorithm that pays close attention to these matters was developed in the 1960s by van Wijngaarden, Dekker, and others at the Mathematical Center in Amsterdam, and later improved by Brent [1]. For brevity, we refer to the final form of the algorithm as *Brent's method*. The method is *guaranteed* (by Brent) to converge, so long as the function can be evaluated within the initial interval known to contain a root.

Brent's method combines root bracketing, bisection, and *inverse quadratic interpolation* to converge from the neighborhood of a zero crossing. While the false-position and secant methods assume approximately linear behavior between two prior root estimates, inverse quadratic interpolation uses three prior points to fit an inverse quadratic function (x as a quadratic function of y) whose value at $y = 0$ is taken as the next estimate of the root x . Of course one must have contingency plans for what to do if the root falls outside of the brackets. Brent's method takes care of all that. If the three point pairs are $[a, f(a)]$, $[b, f(b)]$, $[c, f(c)]$, then the interpolation formula (cf. equation 3.2.1) is

$$x = \frac{[y - f(a)][y - f(b)]c}{[f(c) - f(a)][f(c) - f(b)]} + \frac{[y - f(b)][y - f(c)]a}{[f(a) - f(b)][f(a) - f(c)]} + \frac{[y - f(c)][y - f(a)]b}{[f(b) - f(c)][f(b) - f(a)]} \quad (9.3.1)$$

Setting y to zero gives a result for the next root estimate, which can be written as

$$x = b + P/Q \quad (9.3.2)$$

where, in terms of

$$R \equiv f(b)/f(c), \quad S \equiv f(b)/f(a), \quad T \equiv f(a)/f(c) \quad (9.3.3)$$

we have

$$\begin{aligned} P &= S [T(R - T)(c - b) - (1 - R)(b - a)] \\ Q &= (T - 1)(R - 1)(S - 1) \end{aligned} \quad (9.3.4)$$

In practice b is the current best estimate of the root and P/Q ought to be a “small” correction. Quadratic methods work well only when the function behaves smoothly; they run the serious risk of giving very bad estimates of the next root or causing machine failure by an inappropriate division by a very small number ($Q \approx 0$). Brent’s method guards against this problem by maintaining brackets on the root and checking where the interpolation would land before carrying out the division. When the correction P/Q would not land within the bounds, or when the bounds are not collapsing rapidly enough, the algorithm takes a bisection step. Thus, Brent’s method combines the sureness of bisection with the speed of a higher-order method when appropriate. We recommend it as the method of choice for general one-dimensional root finding where a function’s values only (and not its derivative or functional form) are available.

```
template <class T> roots.h
Doub zbrent(T &func, const Doub x1, const Doub x2, const Doub tol)
Using Brent's method, return the root of a function or functor func known to lie between x1
and x2. The root will be refined until its accuracy is tol.
{
    const Int ITMAX=100; Maximum allowed number of iterations.
    const Doub EPS=numeric_limits<Doub>::epsilon();
    Machine floating-point precision.
    Doub a=x1,b=x2,c=x2,d,e,fa=func(a),fb=func(b),fc,p,q,r,s,tol1,xm;
    if ((fa > 0.0 && fb > 0.0) || (fa < 0.0 && fb < 0.0))
        throw("Root must be bracketed in zbrent");
    fc=fb;
    for (Int iter=0;iter<ITMAX;iter++) {
        if ((fb > 0.0 && fc > 0.0) || (fb < 0.0 && fc < 0.0)) {
            c=a; Rename a, b, c and adjust bounding interval
            fc=fa; d.
            e=d=b-a;
        }
        if (abs(fc) < abs(fb)) {
            a=b;
            b=c;
            c=a;
            fa=fb;
            fb=fc;
            fc=fa;
        }
        tol1=2.0*EPS*abs(b)+0.5*tol; Convergence check.
        xm=0.5*(c-b);
        if (abs(xm) <= tol1 || fb == 0.0) return b;
        if (abs(e) >= tol1 && abs(fa) > abs(fb)) {
            s=fb/fa; Attempt inverse quadratic interpolation.
            if (a == c) {
                p=2.0*xm*s;
                q=1.0-s;
            } else {
                q=fa/fc;
                r=fb/fc;
                p=s*(2.0*xm*q*(q-r)-(b-a)*(r-1.0));
                q=(q-1.0)*(r-1.0)*(s-1.0);
            }
            if (p > 0.0) q = -q; Check whether in bounds.
            p=abs(p);
```